

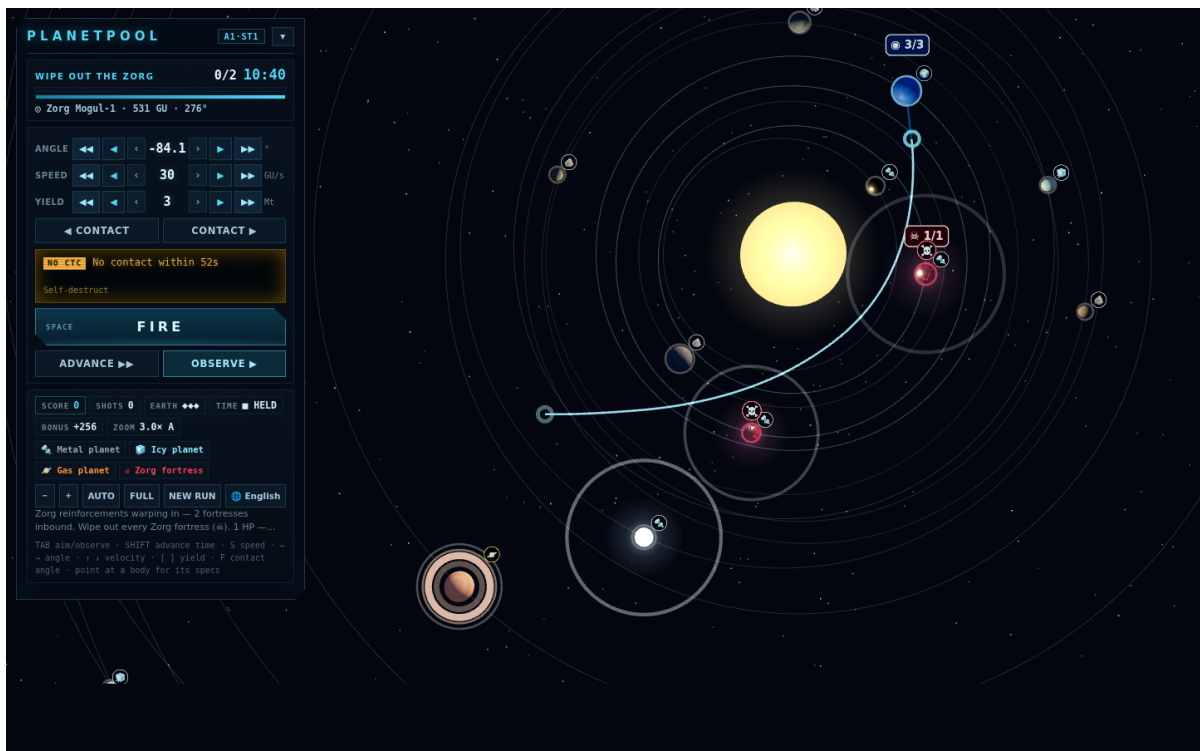
PLANETPOOL × Codex

행성당구 로그라이크 — AI 활용 기록

세 줄 요약

- ① 실제 N체 중력 시뮬레이션 위에서 도는 행성당구 게임입니다. 궤도역학 지식 없이는 만들 수 없는 장르입니다.
- ② 그 지식의 조사부터 코드 작성, 반복 작업까지 구현 전부를 Codex가 맡았습니다. 손으로 만든 예셋은 없습니다 — 그래픽은 코드, 효과음은 합성, 음악은 AI 생성입니다.
- ③ 사람은 아이디어와 목표 경험을 정하고, 매번 직접 플레이하며 재미를 판정했습니다. 이것이 사람이 해야만 하는 일이었습니다.

PLANETPOOL은 핵미사일을 행성의 중력에 태워 휘어 쏘는 우주 당구 로그라이크입니다. 모든 행성이 뉴턴 중력으로 서로를 끌어당기며 궤도를 바꾸고, 직선으로 쏘면 지고 궤적을 휘어야 이깁니다. 이 문서는 게임 자량이 아니라, 이 게임을 만드는 데 AI를 어디에 썼고 사람이 무엇을 했는지에 대한 기록입니다.



실제 플레이 화면. 예측선(하늘색)이 태양 중력에 붙잡혀 휘어 들어간다 — 직선 조준으로는 닿지 않는 판이다. 왼쪽은 LCD풍 사격 통제 패널(각도·속도·탄두 위력·교신 기록).

기술 스택	바닐라 JavaScript(ES 모듈) + three.js · 빌드 도구 없이 정적 호스팅만으로 배포 · 5개 언어 지원
물리	뉴턴 중력 + 소프트닝, KDK 리프프로그(심플렉틱) 적분, 스윙바이 판정, 임펄스/파쇄 모델
오디오	효과음은 ZzFX로 코드 합성(Codex), 음악은 SUNO로 생성한 관현악(사람이 선곡) — 녹음·구매한 오디오 파일 없음

1. AI(Codex)가 한 일

사람이 직접 했다면 각각이 학습, 설계, 혹은 수작업 반복이 필요했을 일들입니다.

영역	내용
기술 선정	활용 라이브러리와 배포 방식을 탐색·비교해 선정
도메인 지식	중력 계수, 탈출 속도, 행성 질량, 공전 속도, N체 문제의 시뮬레이션 방법 등 천문학 여러 분야의 지식을 검색·취합
설계와 구현	주어진 제약 안에서 가장 알맞은 프로그래밍 패턴과 코드 구조를 선정·설계하고 코드를 작성
생성 규칙 고안	판을 동적으로 만들 때의 규칙 — 궤도에 행성을 배치하는 간격, 행성 지름·질량 값의 범위, 미사일 발사 속도의 하한, 요새가 레이저·미사일을 피하는 회피 추진기의 세기와 분사 시점 — 를 천문학 근거를 들어 설계
최적화	궤도·중력 연산 알고리즘 최적화 (예: 예측선 계산의 객체 할당 제거)
코드 정리	폴더 구조와 코드 리팩토링, 쓸모없어진 코드 탐색과 제거
UI 조정	반복 실행하며 UI 문구·배치·색·크기 조정
단순 반복 구현	인트로·튜토리얼 시퀀스, 지시에 따른 비주얼 이펙트 추가와 다듬기 — 깊은 논리 없이 손만 많이 가는 코드
효과음 부착	모든 주요 인게임 상호작용에 효과음을 빠짐없이 부착
다국어	전체 문안 번역과 5개 언어 대응

2. 왜 이 일들은 사람보다 AI가 나은가

- 지식의 문턱이 사라집니다. 궤도역학과 수치 적분을 책으로 익히면 몇 주에서 몇 달이 걸리고, 익혀도 그것을 게임 코드로 옮기는 일이 남습니다. AI는 검색·취합·구현을 한 호흡에 합니다. 어떤 지식이 어디에 쓰였는지는 아래 표에 정리했습니다.
- 단순 반복 작업은 시간 단축보다 휴먼 에러 방지가 큼니다. 효과음을 모든 상호작용에 붙이고 문안을 5개 언어로 옮기는 일에서 사람의 진짜 약점은 속도가 아니라 누락과 오타입니다. AI는 대상 목록을 만들어 전부 처리하므로 빠뜨림이 구조적으로 생기지 않습니다.
- 콘텐츠를 만드는 규칙까지 설계합니다. 판을 절차 생성하려면 "행성을 어디에 얼마나 크게 두는가"부터 "요새가 미사일을 피할 때 추진기를 얼마나 세게 태우는가"까지 전부 규칙이 필요합니다. 궤도 배치는 힐 반경 기반 간격으로 잡고, 질량은 범위를 정한 뒤 지름을 질량에서 파생했으며, 회피 분사는 궤도가 망가지지 않는 세기 상한과 분사 시점(총돌 22초 전)까지 수치로 계측해 규칙으로 만들었습니다. 규칙이 천문학적으로 그럴듯해야 판이 그럴듯해지는데, 이는 전공 지식 없이는 감으로도 못 만드는 부분입니다.
- 정리에 미련이 없습니다. 사람은 자기가 짠 코드를 지우기를 주저하지만, AI는 쓸모없어진 코드를 찾아 걷어내고 폴더 구조를 통째로 재편하는 일을 망설임 없이 합니다.

실제 천문학 지식이 게임 규칙이 된 예

천문학 지식	게임에서 하는 일
심플렉틱(리프프로그) 적분 — 태양계 장기 시뮬레이션의 표준 적분법	N체 엔진의 심장. 널리 쓰이는 RK4는 에너지 오차가 쌓여 몇 분 뒤 궤도가 무너지므로, 에너지를 보존하는 이 방식을 채택
소프트닝 — N체 시뮬레이션에서 천체가 스칠 때 중력이 무한대로 발산하는 것을 막는 기법	미사일이 행성을 아슬아슬하게 스쳐도 계산이 튀지 않게 함
탈출 속도 — 지구에서는 초속 11.2km. 이보다 느리면 되떨어진다	미사일 발사 속도의 하한을 지구 탈출 속도 위로 규정 — 쏘자마자 지구로 되떨어지는 탄이 나오지 않음. "세게 맞은 행성이 계 밖으로 날아가는가"의 판정에도 같은 식 사용
중력 도움(스윙바이) — 보이저 탐사선이 목성의 중력으로 가속해 외행성계까지 간 방법	게임의 핵심 규칙. 행성 근접 통과 거리와 궤적이 꺾인 각도(굴절각)로 "잘 친 샷"을 판정하고 점수 배율을 부여
힐 반경 — 천체가 제 위성을 붙들어 둘 수 있는 중력 지배 영역	행성 궤도 배치 간격의 기준. 서로의 힐 반경을 침범하지 않게 놓아, 생성된 판이 시작하자마자 스스로 붕괴하지 않음
충돌 회피 기동 — 인공위성이 파편을 피할 때 쓰는 짧은 추진기 분사(Δv)	요새·지구의 회피 추진기 규칙. 분사 세기를 궤도 속도의 12% 수준으로 제한 — 계측 결과 56%까지 올리면 회피가 아니라 궤도 이탈이 되고, 근일점이 태양까지 떨어져 "피하려다 타 죽는" 궤도가 나옴
질량-반지름 관계 — 행성의 덩치는 질량에서 따라 나온다	질량 값의 범위만 정하면 지름·내구도가 질량에서 파생 — 목성급은 목성답게 크고 무겁게 보이도록 비율 유지

3. 사람이 한 일

- 게임 아이디어를 고안하고, "중력으로 휘어 치는 당구"라는 목표 경험과 의도를 확정
- 개발 환경을 만들고 Codex를 연결하고, 사용 비용을 결제
- 매 반복마다 직접 플레이하며, 머릿속 그림과 결과물의 차이를 말로 전달 — 개발 기간의 대부분은 이 피드백 순환이었습니다
- 음악은 SUNO로 은하영웅전설풍 관현악을 생성해, 후보를 듣고 고르고 게임에 삽입 — 생성은 AI, 취향의 선택은 사람

4. 사람이 해야만 했던 일

재미의 판정입니다. AI도 Playwright 같은 도구로 게임을 실행하고 화면을 확인할 수 있습니다. 그러나 그 관찰은 스크린샷 단위로 끊겨 있어 입력과 반응 사이의 연속적인 감각이 없고, 화면·소리·손맛을 동시에 느끼는 멀티모달 경험도 아니며, 수십 년 게임을 해 온 사람의 축적된 감식안도 없습니다. "이 판이 재미있는가, 머릿속 그림과 같은가"는 사람이 직접 플레이해야만 답할 수 있었습니다.

그리고 의도의 소유입니다. 무엇을 만들지, 어떤 경험을 지킬지, 어디서 멈출지는 AI가 정해 주지 않습니다. AI는 시킨 일을 잘하지만, 무엇을 시킬지 정하는 것은 끝까지 사람의 일이었습니다.

이 게임에서 AI는 지식·구현·반복을 맡았고, 사람은 의도·판정·선택을 맡았다. 전공 지식이 필요한 장르를 비전공자가 완성한 것이 이 분업의 결과다.